BCSE497J - Project-I

CONTEXT-AWARE PERSONAL AI ASSISTANT USING LARGE LANGUAGE MODELS, DYNAMIC MODEL ROUTING, MEMORY, AND RETRIEVAL-AUGMENTED GENERATION (RAG)

23BAI0111	**NISCHAY N**
23BAI0105	**TEJAS K M**
23BCT0227	**TENIDHAR REDDY**

Under the Supervision of

Prof. Mary Mekala A

School of Computer Science and Engineering (SCOPE)

B.Tech.

in

Computer Science and Engineering

School of Computer Science and Engineering



November 2026

ABSTRACT

Assistants such as ChatGPT, Claude and Gemini leave the choice of model to the user, and products that automate the choice, such as Perplexity, do not disclose how they choose. This project builds an open-source, local-first personal assistant that sends each query to the model **measured** to be best for it, remembers the user across sessions, and answers from the user's own documents. Any model can be plugged in through a registry file: local models served by Ollama or any cloud API. A calibration step runs each model on 200 tasks drawn from public benchmarks written outside the team (MBPP, GSM8K, Natural Questions, IFEval and generated images) and writes a scorecard of accuracy per task type with a 95% Wilson confidence interval, median latency on the user's machine, and cost per query computed from the provider's own token counts. A transparent Routing Utility Score (RUS) combines the lower confidence bound of accuracy with cost and latency under user-set weights; no input to it is estimated by hand. Memory, the document index, embeddings and scorecards stay on the user's computer, prompts that carry personal context are routed only to local models, and every request that leaves the device is written to an audit log.

On 200 unseen test tasks (disjoint from the calibration tasks), with four local models and three cloud models (gemini-3.1-flash-lite, claude-haiku-5.5, claude-sonnet-5.5) on an 8 GB Apple M3 laptop, the routed system answered 89.5% correctly (95% CI 84% to 93%) under balanced weights, level with always using claude-sonnet-5.5 (89.0%) at 95% lower measured cost, and ahead of the best local model (48.0%). Because small cloud models are now cheap and far more accurate than 1–3B local models, the score alone sends no traffic to the local models; a local-first cascade that keeps a local answer only when a reference-free check passes (the prompt's own example test for code, agreement between two local models for arithmetic) answered 28.5% of queries on the device at 88.0% accuracy (CI 83% to 92%) and 96% lower cost than claude-sonnet-5.5. An ablation over five builds shows memory raising cross-session recall from 0/30 to 30/30 (95% CI 89%–100%) and retrieval raising closed-domain accuracy from 0% to 86% (CI 71%–94%); with every cloud model available, 0 prompts carrying personal context left the device. Every rate is reported with its sample size and confidence interval, and every number traces to a logged answer.

Keywords — Dynamic Model Routing, Large Language Models, Retrieval-Augmented Generation, Persistent Memory, Local-First AI, Cost-Aware Inference, Ablation Study.

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	i
1.	**INTRODUCTION**	1
	1.1 Background	1
	1.2 Motivation	1
	1.3 Scope of the Project	1
2.	**PROJECT DESCRIPTION AND GOALS**	2
	2.1 Literature Review	2
	2.2 Research Gap	2
	2.3 Objectives	3
	2.4 Problem Statement	3
	2.5 Project Plan	4
3.	**TECHNICAL SPECIFICATION**	5
	3.1 Requirements	5
	3.2 Feasibility Study	5
	3.3 System Specification	6
4.	**DESIGN APPROACH AND DETAILS**	7
	4.1 System Architecture	7
	4.2 Design	7
	4.3 Scoring and Routing	14
5.	**IMPLEMENTATION AND RESULTS**	16
	5.1 Implementation	16
	5.2 Experimental Setup	16
	5.3 Results	19
	5.4 Prototype Demonstration	26
	5.5 Discussion	33
	5.6 Limitations and Future Work	35
6.	**REFERENCES**	36

1. INTRODUCTION

1.1 Background

Large language models differ in what they are good at. A code-tuned model can write correct functions yet fail simple arithmetic word problems; a large hosted model can answer factual questions that a small local model gets wrong; only some models accept images at all. Using the best model for each task today means a person judging the task and switching tools by hand, losing the conversation and anything they told the previous assistant. Products that automate this choice exist. Perplexity AI routes across several commercial models, keeps memory and grounds answers in retrieved sources, but its routing rule is undisclosed, every query is sent to the provider's servers, and the contribution of each component has not been published.

A second pressure is cost. Commercial APIs charge per token, and a capable model is far more expensive than a small open-weight model running on a laptop for free. Much of what people ask, however, does not need the most expensive model. A router that knows, from measurement, which tasks a free local model handles well can keep those tasks local and pay for a cloud model only where it is measurably better.

1.2 Motivation

The motivation is threefold. First, routing decisions should be explainable: a user, an examiner or a developer should be able to see exactly why a model was chosen, with every input to the decision measured rather than assumed. Second, personal data should stay personal: memory of the user and their private documents should live on their own computer, and a cloud model should see only what a given query requires. Third, claims about such a system should be measured with the statistical care they need: small benchmarks produce wide uncertainty, and a reported 100% from ten questions is not evidence of perfection.

At Review 2 this project was criticised on exactly these points: the router's accuracy inputs were hand-entered estimates, the scoring was not defined well enough to say what a score meant, the evaluation prompts were written by the team and produced 100% results without confidence intervals, and the diagrams in the report did not match the code. This report rebuilds the routing on measured scorecards, evaluates it on public benchmarks with a held-out split, defines every metric (Section 4.3 and Section 5.2.4), and generates every chart and diagram from code.

1.3 Scope of the Project

The project covers the design, implementation and evaluation of an assistant with four components: (A) dynamic model routing driven by an intent classifier and the Routing Utility Score; (B) persistent memory; (C) retrieval-augmented generation over the user's documents; and (D) their combination. It is released as open-source software: any model can be registered and calibrated, and the user controls the weights that trade accuracy against cost and latency. The evaluation covers routing against every single model on public benchmarks, measured API cost, an ablation of the components, and a privacy check of what leaves the device. Live web search and very large local models are outside the scope; the reference hardware is an 8 GB laptop.

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

The review covers the three pillars of the system and, since Review 2, the work on cost-aware model routing that the revised routing builds on.

Table 1. Related work

Work	Objective	Method	Pros / Cons	Findings
Lewis et al., RAG (NeurIPS 2020) [1]	Ground answers in retrieved documents	Dense retriever + seq2seq generator	+ Cuts hallucination / – no routing or memory	RAG beats parametric-only models on open QA
Yao et al., ReAct (ICLR 2023) [2]	Interleave reasoning and tool use	Thought / action steps in the prompt	+ Multi-step tasks / – no persistent memory	Reasoning with acting beats either alone
Packer et al., MemGPT (2023) [3]	Long-term memory for LLMs	OS-style paging between memory tiers	+ Long-term memory / – no routing	Coherent chats beyond the context limit
Chen et al., FrugalGPT (2023) [4]	Cut LLM cost without losing quality	Cascade: cheap model first, a scorer decides whether to escalate	+ Large cost savings / – learned scorer, cloud-only, no memory	Matches the best model at a fraction of the cost on several datasets
Ding et al., Hybrid LLM (ICLR 2024) [5]	Route between a small and a large model	Router trained to predict the quality gap per query	+ Quality-aware / – two models, trained router	Fewer calls to the large model at small quality loss
Ong et al., RouteLLM (2024) [6]	Learn when a strong model is needed	Routers trained on human preference data	+ Strong cost-quality trade-off / – opaque learned router, two models	Large cost reduction with little quality loss on benchmarks
Perplexity AI, production system [7]	Search-grounded answers	Classifier routing across commercial models + hybrid RAG	+ Proven at scale / – undisclosed router, cloud-only, no ablation	All three pillars viable in production

FrugalGPT, Hybrid LLM and RouteLLM show that routing can cut cost substantially, but each relies on a router learned from data that a user cannot inspect or adjust, and each treats routing between one cheap and one strong cloud model. None keeps the user's data on the device, and none combines routing with memory and retrieval in one measured system.

2.2 Research Gap

At Review 1 the claimed novelty was that no system combines routing, memory and RAG; the study of Perplexity showed that claim no longer holds. What remains open, and what this project addresses, is:

1. **A transparent, measured routing rule.** A closed-form score whose every input (accuracy with its uncertainty, cost per query, latency) is measured on public tasks and shown with each decision, instead of hand-set priors or an opaque learned router.

2. **Local-first routing with a privacy constraint.** Routing across local and cloud models in which private context is a hard eligibility rule, with an audit of every request that leaves the device.
3. **An open, pluggable system.** Any model, local or hosted, is added by configuration and calibrated with one command, so the routing table reflects the user's own models and hardware.
4. **An ablation with honest statistics.** The contribution of routing, memory and retrieval measured separately, with confidence intervals and paired significance tests.

2.3 Objectives

1. Build a model registry and calibration step that produces, for any model, a scorecard of accuracy per task type (with 95% confidence intervals), latency and cost per query. **Status: achieved** (7 models calibrated on 200 public tasks).
2. Define and publish a routing score that uses only measured inputs, and show every decision. **Status: achieved** (Routing Utility Score, Section 4.3).
3. Evaluate routing on held-out public tasks against every single model, with significance tests and measured cost. **Status: achieved** (200 test tasks, Section 5.3).
4. Cut paid API usage without losing accuracy. **Status: achieved** (95% lower cost than claude-sonnet-5.5 at equal accuracy; 28.5% of queries answered on the device with the local-first cascade).
5. Keep personal data on the device and prove it. **Status: achieved** (private-context egress in the privacy check: 0 requests).
6. Measure memory and retrieval in a five-build ablation. **Status: achieved** (Section 5.3.6).

2.4 Problem Statement

Users must choose a model per task by hand, lose context when they switch, and pay cloud prices for tasks a free local model could handle; the systems that automate the choice send all data to the cloud and decide with rules no one can inspect. The problem addressed is to build and evaluate an assistant that routes each query using measured, inspectable evidence, keeps personal context on the user's computer, reduces paid API usage where that costs little accuracy, and quantifies what each component contributes.

2.5 Project Plan

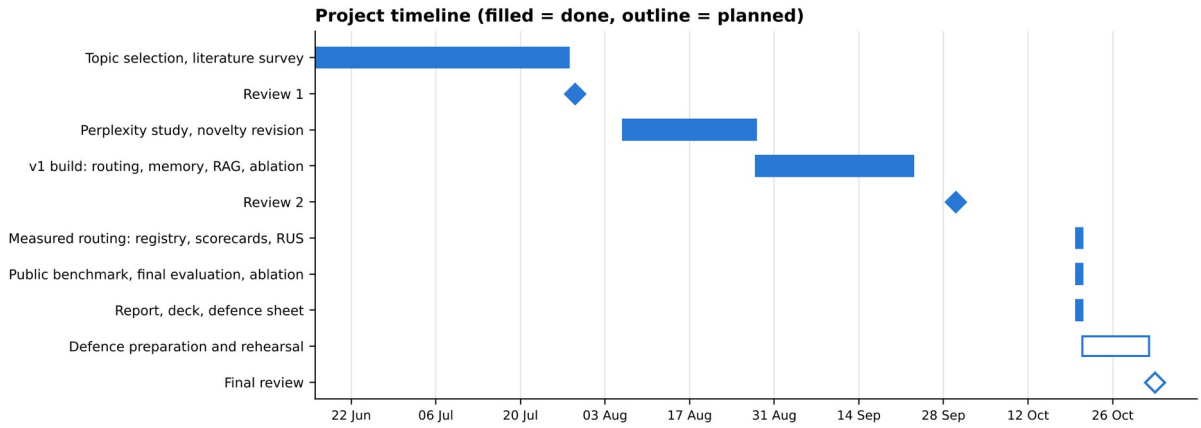


Fig. 1. Project timeline from the team's records (filled = done, outline = planned)

3. TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional

- Register any model (local or cloud) in a configuration file and calibrate it with one command.
- Classify each query into a task type: coding, writing, search (factual), reasoning or multimodal.
- Score every eligible model with the Routing Utility Score and answer with the highest-scoring one.
- Let the user set the accuracy, cost and latency weights, and show the full scoring table for each decision.
- Store user facts, session summaries and turns locally and recall them in later sessions (Model B).
- Retrieve passages from the user's documents and answer with numbered citations (Model C).
- Keep prompts that carry memory or private passages on local models by default; log every cloud request.
- Record the cost of every answer from the provider's token counts.

3.1.2 Non-Functional

- **Transparency:** every routing input is measured and every decision is inspectable and reproducible.
- **Privacy:** personal data stays in a local folder; the audit log stores no prompt text.
- **Extensibility:** a new model or provider needs a configuration entry, not code changes.
- **Reproducibility:** fixed seeds, temperature 0, every answer and grade saved; results recomputable without calling a model.
- **Performance:** classification and routing take milliseconds; generation time is reported per model.
- **Reliability:** a failing or rate-limited provider is retried, then reported, without stopping the session.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The system uses standard open libraries (scikit-learn, NumPy, pandas, SQLite, Streamlit, Matplotlib, PyYAML) and Ollama for local inference. The intent classifier trains in seconds on a CPU, and local 1B–3B models run on an 8 GB laptop. Cloud providers are reached over plain HTTPS without vendor SDKs.

3.2.2 Economic Feasibility

Local inference is free. gemini-3.1-flash-lite was used through Google's free tier at no cost. The two Claude models were paid from prepaid API credit with a hard spending cap of $5 in the

code; the whole evaluation, including retries and the repeat run, spent $1.87. All cost figures in Chapter 5 use each provider's published paid rates, so the savings reflect what a paying user would see.

3.2.3 Social Feasibility

Keeping personal context on the user's device addresses a common objection to AI assistants. The open-source release and the published scoring rule let anyone audit how the assistant decides, and the egress log lets a user verify what was sent to a provider.

3.3 System Specification

3.3.1 Hardware Specification

- Processor: any modern multi-core CPU; evaluation on an Apple M3 laptop without a discrete GPU.
- Memory: 8 GB minimum for 1B–3B local models; 16 GB for 7B–8B models.
- Storage: about 8 GB for the local model weights; under 100 MB for code, data and results.

3.3.2 Software Specification

- Operating system: macOS or Linux (Windows through WSL).
- Language: Python 3.10+; Ollama runtime for local models.
- Libraries: scikit-learn, pandas, NumPy, joblib, PyYAML, requests, Streamlit, Matplotlib, pytest; Graphviz for diagrams.
- Storage: SQLite for memory; a pickled embedding index for retrieval; JSON scorecards; JSON Lines logs.
- Optional cloud APIs: Gemini, any OpenAI-compatible endpoint, Anthropic.

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

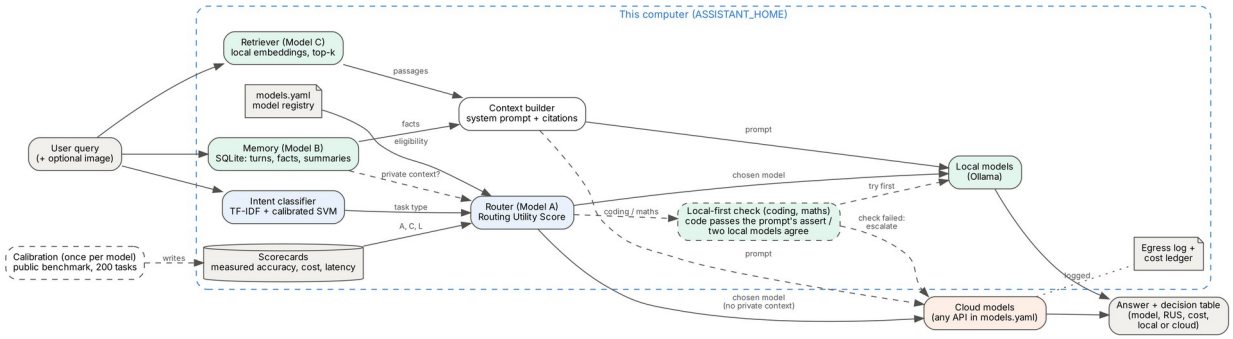


Fig. 2. System architecture. Everything inside the dashed box runs on the user's computer

A query is first matched against memory and the user's documents, then classified. The router scores every eligible model with the Routing Utility Score, using the measured scorecards and the model registry; if memory or retrieval contributed private context, only local models are eligible. For coding and arithmetic the local-first check can answer on the device before any model is called remotely. The context builder places recalled facts and numbered passages into the system prompt, the chosen model answers, and the turn is written back to memory. Calibration, run once per model, is what fills the scorecards.

4.2 Design

Every diagram in this section is generated by a script in the repository (scripts/make*diagrams.py and scripts/diagrams*uml.py) and follows a standard notation. The class diagrams are produced by parsing the source code, so a class, attribute or operation that does not exist cannot appear; the PlantUML sources are kept in docs/diagrams/src.

4.2.1 Data Flow Diagrams

Yourdon–DeMarco notation: external entities are rectangles, processes are numbered circles, data stores are open-ended boxes with an identifier, and every arrow is a named data flow. Level 0 shows the system as one process with its four external entities; level 1 breaks one query into the eight processes of pipeline.Assistant.chat, with the files each one reads or writes.

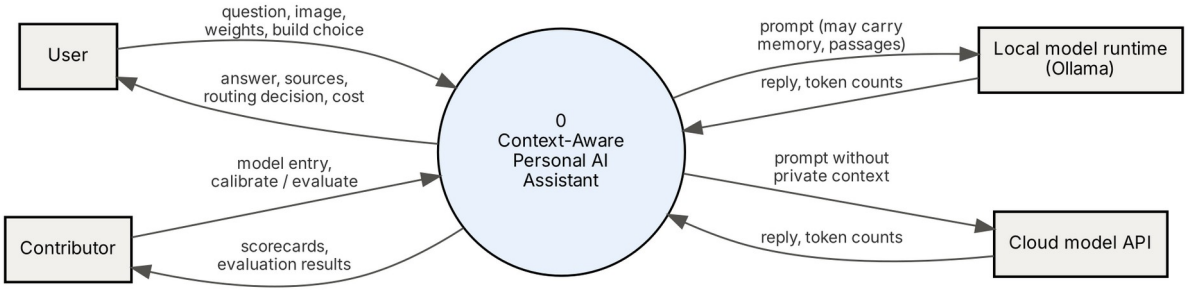


Fig. 3. Data flow diagram, level 0 (context diagram)

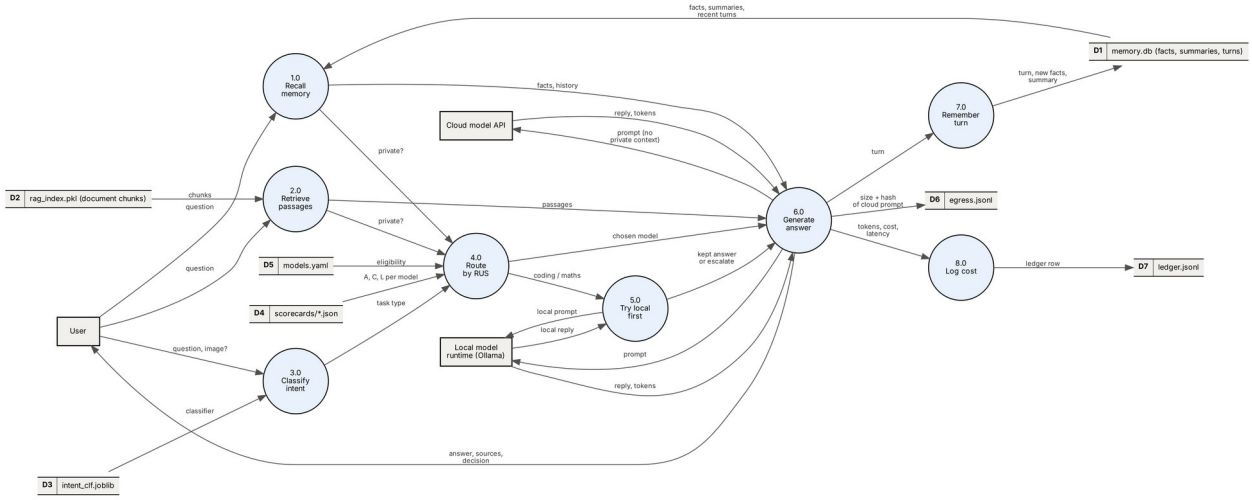


Fig. 4. Data flow diagram, level 1: one query through the full system (Model D). D1–D7 are the real files in config.py

4.2.2 Use Case Diagrams

UML 2.5 notation: actors outside the system boundary, use cases inside it, associations as plain lines, «include» for behaviour that always happens and «extend» for behaviour that happens only under the stated condition. The User's use cases are the controls of the app (and the chat command); the Contributor, a specialised User, registers, calibrates and evaluates models from the command line. The local model runtime and the cloud APIs are secondary actors.

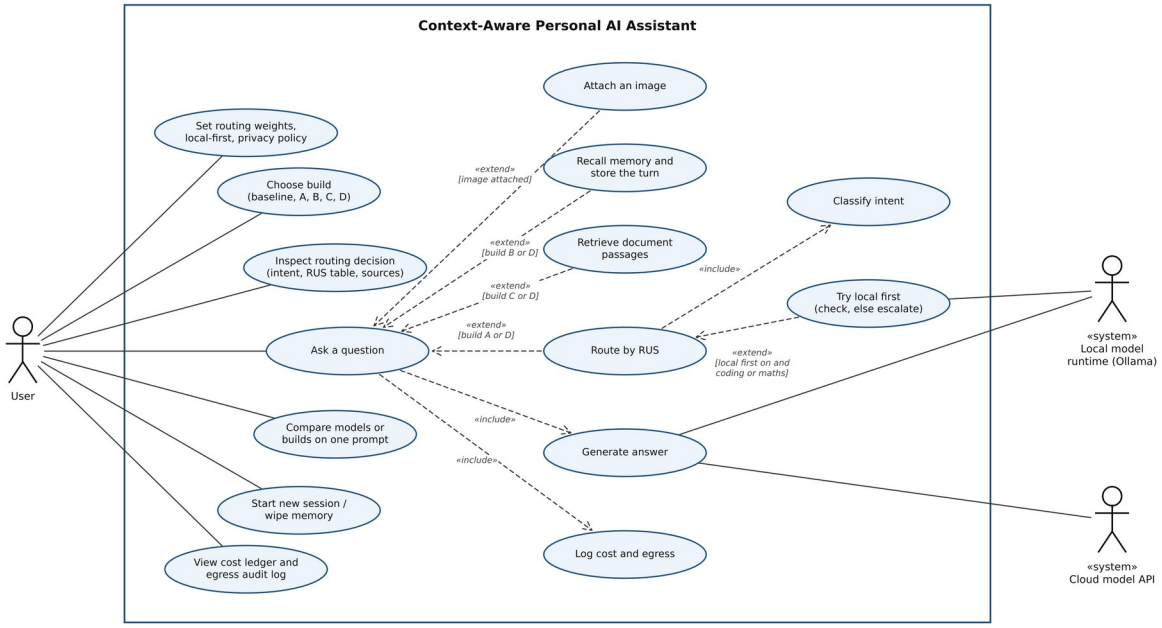


Fig. 5. Use case diagram for the User. «extend» conditions name the build (A–D) or setting that enables the behaviour

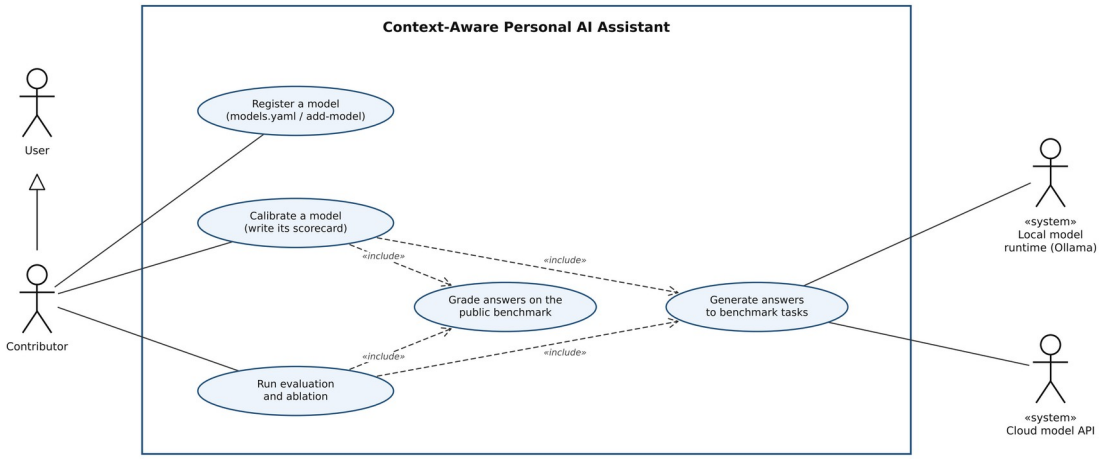


Fig. 6. Use case diagram for the Contributor, who is also a User (generalisation)

4.2.3 Class Diagrams

UML class notation generated from the code: each box lists the attributes with their types and the public operations with their parameters and return types; underlined operations are module-level functions or class methods, shown in classes stereotyped «module». Filled diamonds are composition (the Assistant creates and owns its classifier, memory and retriever, each optional because it depends on a build switch), plain arrows are associations with multiplicities, dashed arrows are dependencies, and the hollow triangle is inheritance.

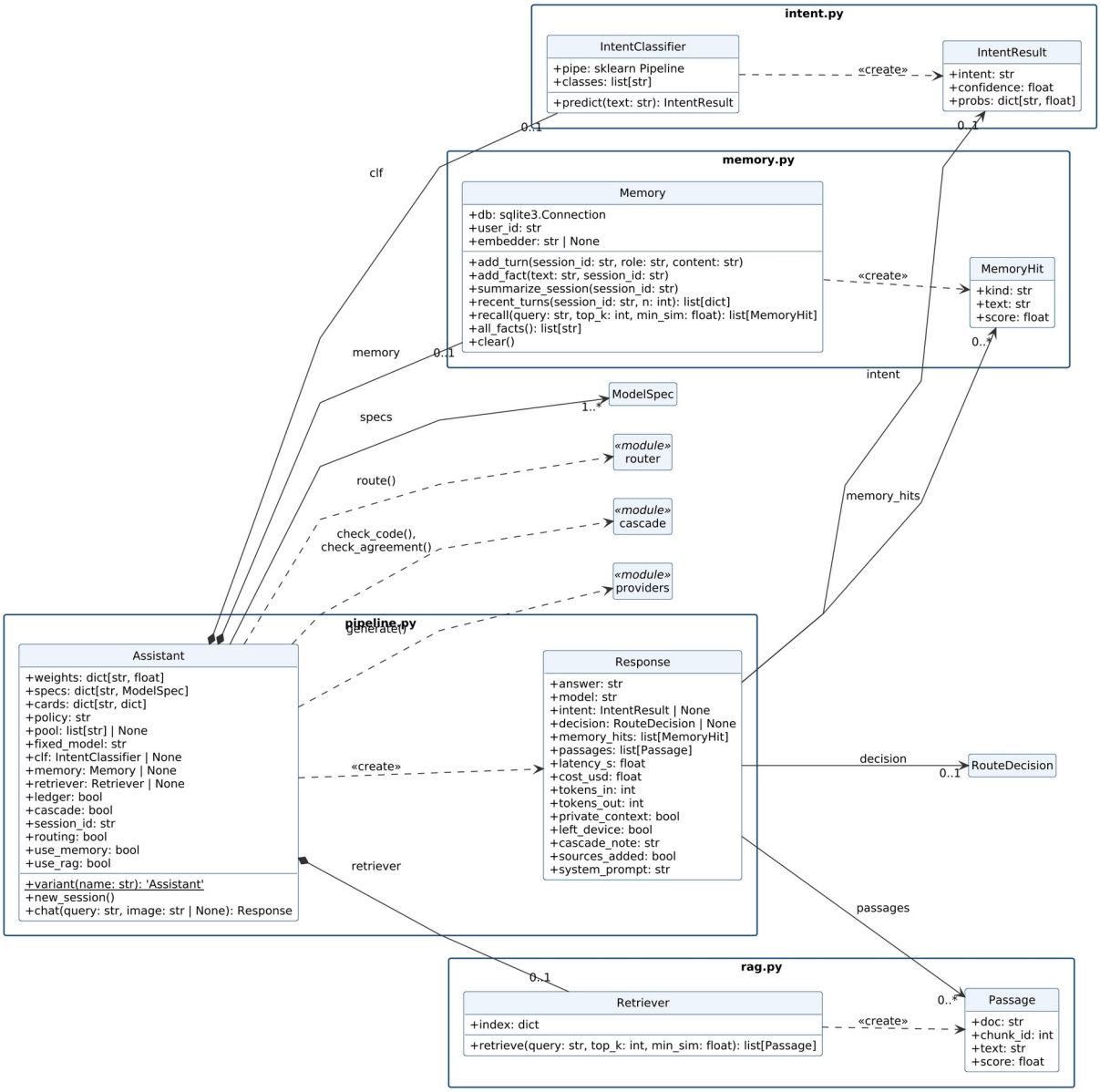


Fig. 7. Class diagram (a): the pipeline and the classes it owns

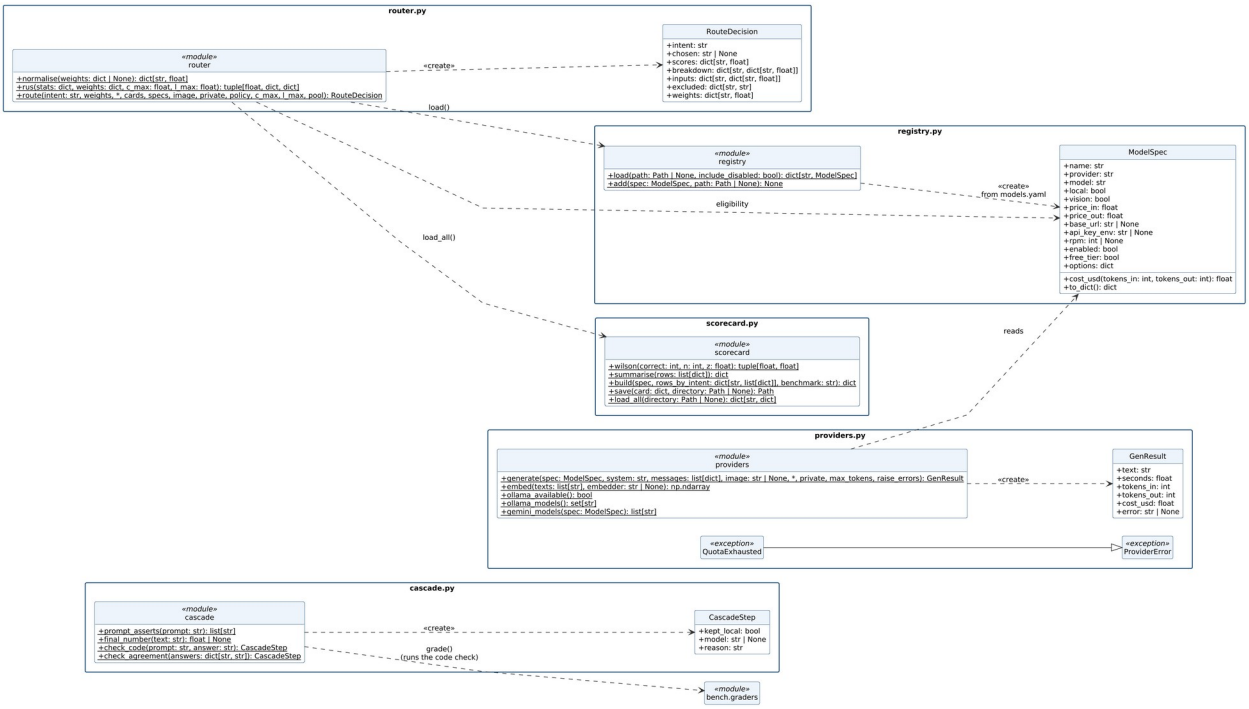


Fig. 8. Class diagram (b): routing, the local-first check, the model registry, scorecards and providers

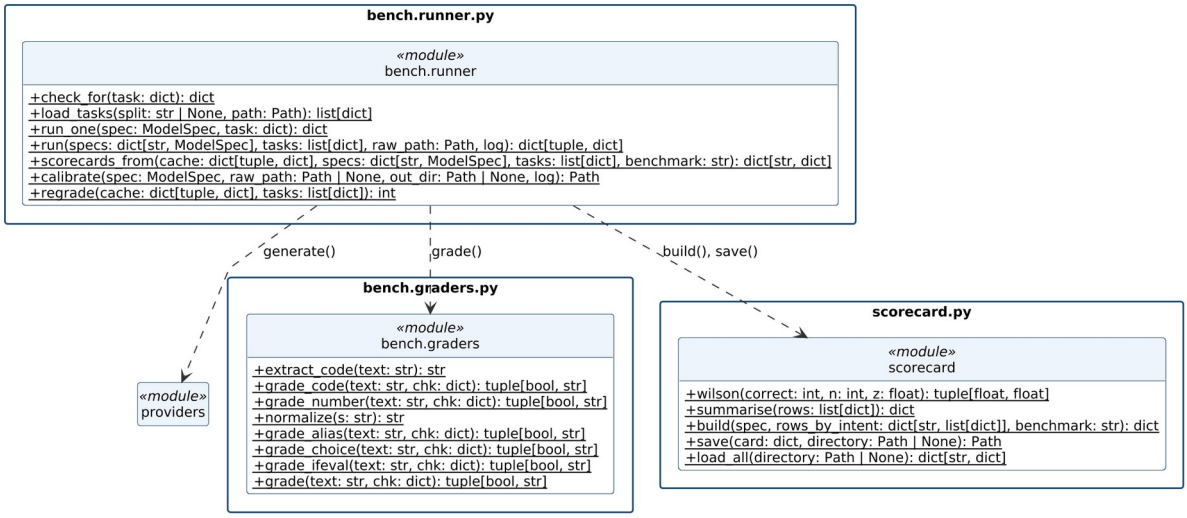


Fig. 9. Class diagram (c): the benchmark runner and graders that produce scorecards

4.2.4 Sequence Diagram

UML sequence notation: lifelines for each object or module in call order, solid arrows for calls, dashed arrows for returns, activation bars while a call is running, and opt fragments for the steps that depend on a build switch or setting.

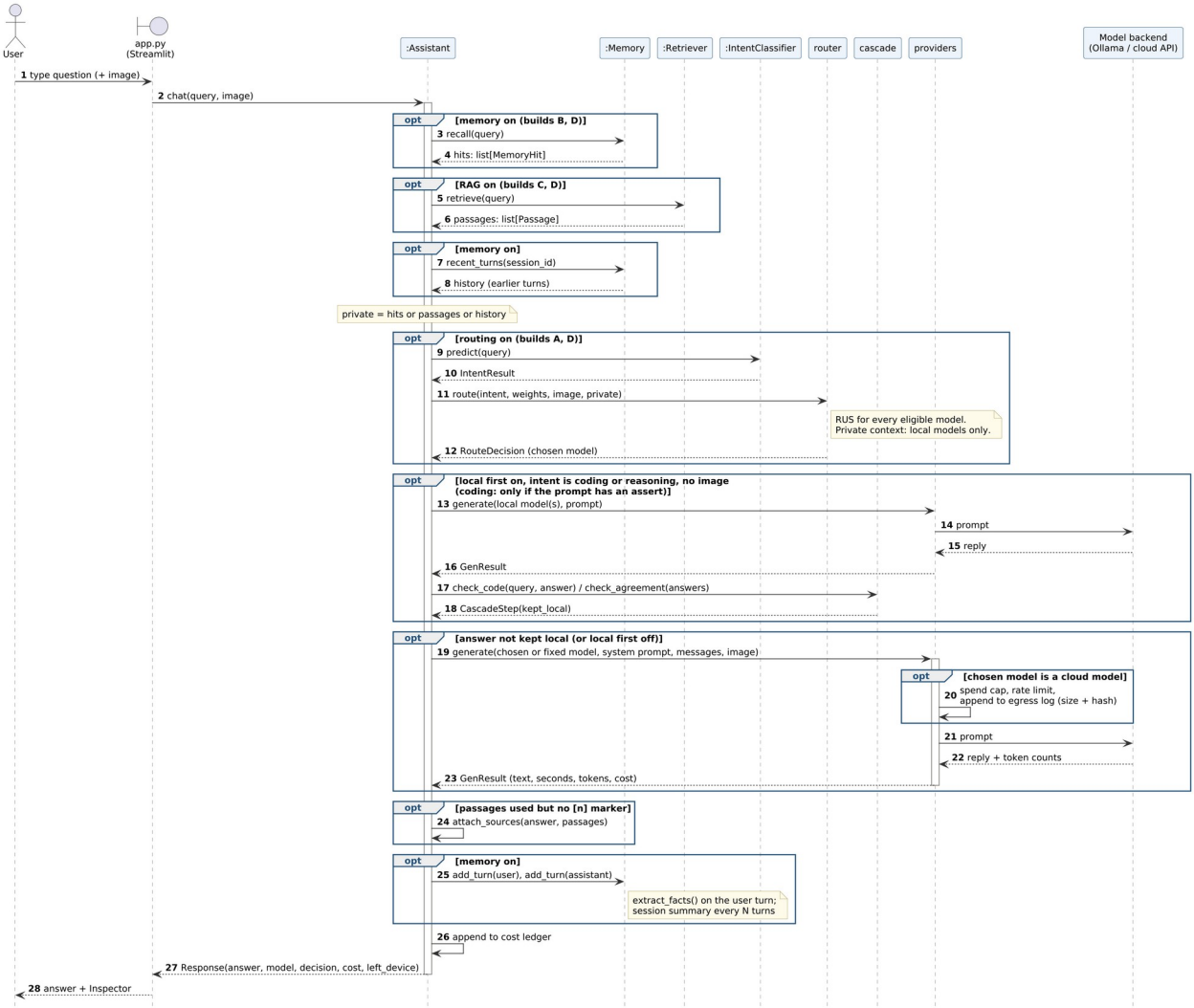


Fig. 10. Sequence diagram: one call to pipeline.Assistant.chat, in the order of the calls in the code

4.2.5 Activity Diagram

UML activity notation with swimlanes for the user, the assistant on the user's computer and the model backend; diamonds are decisions and merges. It shows the same query as the sequence diagram as a flow of decisions, including when the local-first check keeps an answer on the device and when the query escalates.

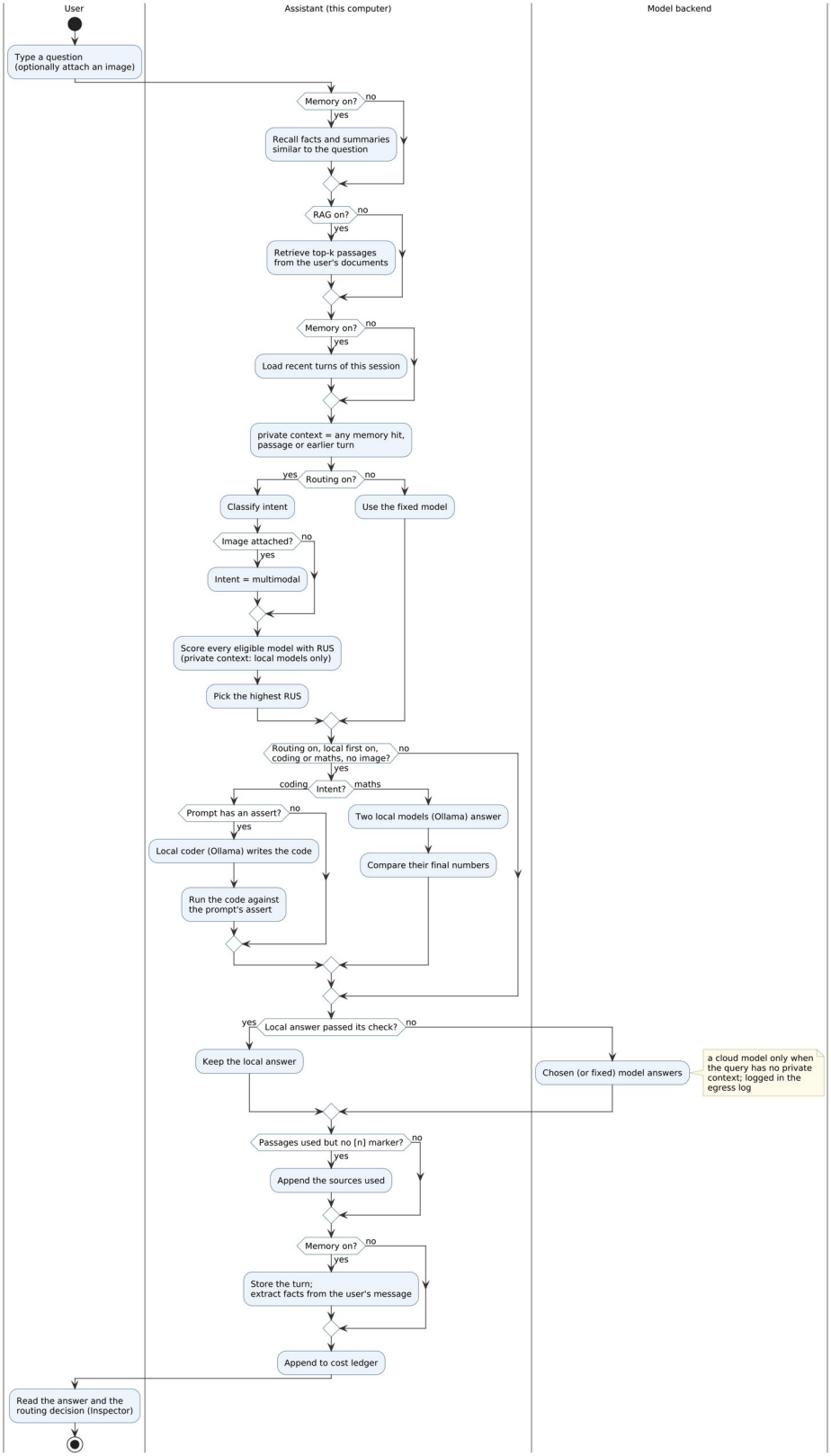


Fig. 11. Activity diagram: how one question is answered

4.3 Scoring and Routing

4.3.1 The Scorecard

Calibration runs a model on the 200-task calibration split (40 tasks per task type) and grades every answer by code. The scorecard stores, for each task type, the number of tasks n, the number correct, the accuracy, the 95% Wilson interval of the accuracy, the median latency in seconds on the machine that ran it, and the mean cost per query in US dollars computed as (input tokens × input price + output tokens × output price) / 10^6 from the provider's own usage report. Local models cost nothing per token.

The Wilson interval is used because a small sample does not establish perfect accuracy: with z = 1.96, ten correct out of ten gives an interval of 72% to 100%, and twenty out of twenty gives 84% to 100%.

$$centre = (p + z^2/2n) / (1 + z^2/n), \quad half\text{-}width = z \cdot \sqrt{(p(1-p)/n + z^2/4n^2)} / (1 + z^2/n)$$

4.3.2 The Routing Utility Score

For a query of task type i, every eligible model m receives

$$RUS(m, i) = w_a \cdot A(m, i) + w_c \cdot (1 - \min(C(m, i)/C_max, 1)) + w_l \cdot (1 - \min(L(m, i)/L_max, 1))$$

Table 2. Definition of every term in the Routing Utility Score

Symbol	Meaning	Source
A(m, i)	Lower bound of the 95% Wilson interval of m's accuracy on task type i	measured (calibration)
C(m, i)	Mean cost per query of m on task type i, USD	measured (token counts × price)
L(m, i)	Median latency of m on task type i, seconds	measured (this machine)
C_max	The most the user will pay for one query	user setting, default $0.002
L_max	The longest the user will wait	user setting, default 30 s
w_a, w_c, w_l	Weights for accuracy, cost and latency, normalised to sum to 1	user setting, default 0.6 / 0.2 / 0.2

Each bracket is a fraction between 0 and 1, so RUS lies between 0 and 1 and reads as a share of an ideal model: one that is certainly always right, free and instant scores 1. Using the lower confidence bound means a model is never credited with more accuracy than its evidence supports. A model is eligible only if it has a scorecard for the task type, accepts an image when the query has one, and runs locally when the prompt carries private context (the default policy). The highest score wins; ties go to the cheaper, then the faster model.

4.3.3 Worked Example

Table 3 shows the decision for a coding query under the default weights, filled in from the measured scorecards. Each row's RUS is the sum of its three terms.

Table 3. Routing Utility Score for a coding query, default weights (chosen model highlighted)

Model	Accuracy (n)	A	C ($/query)	L (s)	$w_a \cdot A$	w_c term	w_l term	RUS
claude-haiku-5.5	98% (39/40)	0.871	0.000132	2.2	0.523	0.187	0.185	**0.895**
gemini-3.1-flash-lite	88% (35/40)	0.739	0.000123	1.3	0.443	0.188	0.191	**0.822**
claude-sonnet-5.5	100% (40/40)	0.912	0.001618	2.1	0.547	0.038	0.186	**0.772**
qwen2.5-coder-3b	55% (22/40)	0.398	0.000000	5.0	0.239	0.200	0.167	**0.606**
llama3.2-3b	50% (20/40)	0.352	0.000000	2.1	0.211	0.200	0.186	**0.597**
llama3.2-1b	45% (18/40)	0.307	0.000000	1.6	0.184	0.200	0.189	**0.574**
moondream-1.8b	8% (3/40)	0.026	0.000000	0.6	0.015	0.200	0.196	**0.411**

5. IMPLEMENTATION AND RESULTS

5.1 Implementation

The system is one Python package with one module per responsibility; all five builds share the same code path and differ only in three switches.

Table 4. Modules

Module	File	Responsibility
Registry	models.yaml, assistant/registry.py	Any model: provider, model id, local or cloud, vision, price per million tokens
Providers	assistant/providers.py	Ollama, OpenAI-compatible, Anthropic, Gemini; token counts and cost; egress audit
Scorecards	assistant/scorecard.py	Per-model measurements per task type; Wilson intervals
Router	assistant/router.py	Routing Utility Score; eligibility (vision, privacy, calibration)
Intent classifier	assistant/intent.py	TF-IDF word 1–2-grams and character 3–5-grams, calibrated linear SVM; 1,350 labelled queries
Memory	assistant/memory.py	SQLite: turns, extracted facts, session summaries; cosine top-k recall
Retrieval	assistant/rag.py	120-word chunks (30 overlap), local embeddings, top-4 passages
Pipeline	assistant/pipeline.py	Assistant(routing, memory, rag); cost ledger
Benchmark	assistant/bench/, scripts/	Graders, calibration, evaluation, ablation, figures, diagrams
Interfaces	app.py, assistant/*_main_*.py	Streamlit app with decision inspector; command line

An offline test-suite of 16 tests (deterministic mock model, no network) checks the Wilson interval, that the score's terms add up and stay in [0, 1], that weights move traffic between free and paid models, that private context never reaches a cloud model, that images reach only vision models, that an uncalibrated model is never chosen, the graders, the benchmark split, memory recall, retrieval and the end-to-end Model D path.

5.2 Experimental Setup

5.2.1 Models

Table 5. Model pool

Model	Where	Images	Price (USD / 1M tokens in, out)
qwen2.5-coder-3b	local (Ollama)	no	free
llama3.2-3b	local (Ollama)	no	free
llama3.2-1b	local (Ollama)	no	free
moondream-1.8b	local (Ollama)	yes	free
gemini-3.1-flash-lite	cloud API	yes	0.25, 1.5
claude-haiku-5.5	cloud API	yes	0.1, 0.5
claude-sonnet-5.5	cloud API	yes	2, 10

Local models run on an Apple M3 laptop with 8 GB of memory. All generation uses temperature 0 so that answers are repeatable.

5.2.2 Benchmark

Table 6. Public benchmark: 80 tasks per type, written outside the team

Task type	Source (authors)	Licence	Graded by
Coding	MBPP, test range (Austin et al.) [8]	CC-BY-4.0	the dataset's own unit tests, executed
Reasoning	GSM8K test set (Cobbe et al.) [9]	MIT	exact final number
Search (factual)	Natural Questions open, dev (Lee et al.) [10]	CC-BY-SA-3.0	gold answer contained after normalisation
Writing	IFEval (Zhou et al.) [11]	Apache-2.0	every verifiable instruction satisfied (strict)
Multimodal	80 generated images: charts, counting, colours, shapes, words	this project	known ground truth

Each task type was shuffled with a fixed seed and split in half: the **calibration split** (200 tasks) is used only to build the scorecards; the **test split** (200 tasks) is used only to evaluate routing. Every model answered every task, so any routing policy is evaluated on the same graded answers.

5.2.3 Policies Compared

- **Always m:** every query to one model, for each model in the pool.
- **Routed:** intent classifier + Routing Utility Score with three weight settings: balanced (0.6/0.2/0.2), quality-first (1/0/0) and cost-saver (0.4/0.5/0.1).
- **Local-first cascade:** for coding, the local coder answers and its code is run against the example `assert` in the user's prompt; for arithmetic, two local models answer and must agree on the final number. A passing answer is kept on the device; otherwise the query goes to the balanced RUS choice. The checks never see a reference answer.
- **Oracle (hindsight):** for each task, the cheapest model that answered it correctly. No real router can do this; it is the upper bound.

5.2.4 Metrics

- **Accuracy:** share of test tasks graded correct, with the 95% Wilson interval.
- **Paired significance:** exact McNemar test on the tasks where exactly one of two policies is right [13].
- **Cost:** measured USD per 1,000 queries (and INR at ₹88 per USD); local models cost nothing per token.
- **Cost reduction:** 1 − cost(routed) / cost(always the best cloud model), always reported with the accuracy difference.
- **Ablation:** memory recall, RAG accuracy, abstention, hallucination (neither correct nor an abstention) and citation rate, each with n and a 95% interval.

5.2.5 Grading Audit

Every grade is produced by code and saved with its reason, so the graders were audited against the saved answers before any result was read off. Three corrections came out of the audit, each applied to every model alike:

- **Code that crashes in its own example.** Some answers ended with a usage example that raised an error, which hid a correct function. The grader now runs the answer, keeps whatever functions it defined, and then runs the dataset's tests.
- **Answers cut off by the output budget.** The Claude models think before answering and some ran out of output tokens before writing an answer. Any answer that used the whole budget and was graded wrong was asked once more with a 4,096-token budget (88 answers; 39 were then correct).
- **Exact-match grading of factual answers.** Natural Questions accepts only its listed spellings, so "May 10, 1940" failed against "10 May 1940" and "Rachel Bilson" against "Rachel Sarah Bilson". The grader now also accepts a word-overlap match: every number in the gold answer must appear, a two-word answer needs its head word, a longer one two thirds of its words, and words already in the question do not count. With the original strict grading the headline does not change: balanced routing 87.5% against 87.5% for claude-sonnet-5.5. Natural Questions answers also date from 2018, so a correct current answer (for example the most recent NBA title) can still be marked wrong; search accuracy is therefore a lower bound for every model.

5.3 Results

5.3.1 Measured Scorecards

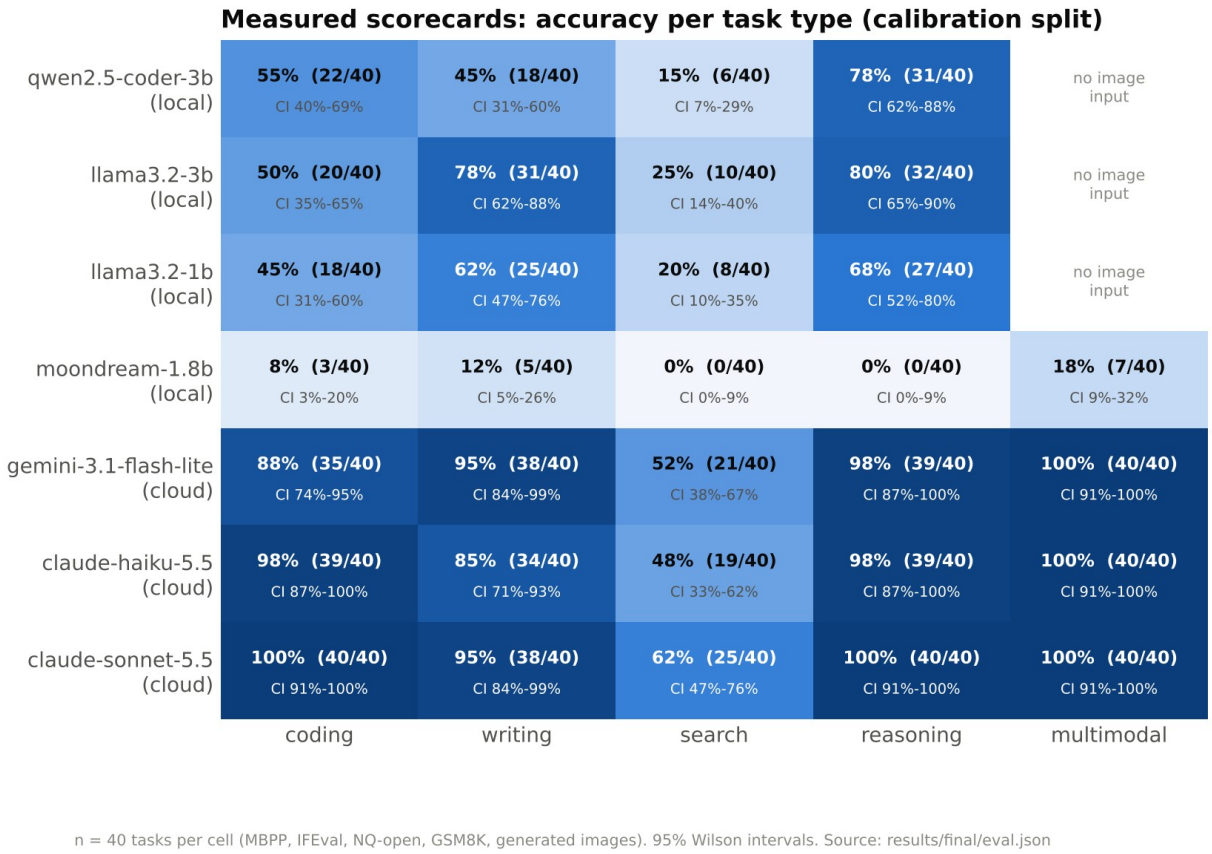


Fig. 12. Measured accuracy per model and task type on the calibration split, with 95% Wilson intervals

Table 7. Median latency per task type on the reference laptop, and mean measured cost per 1,000 queries

Model	Where	coding	writing	search	reasoning	multimodal	Mean $ / 1k
qwen2.5-coder-3b	local	5.0 s	2.4 s	0.8 s	10.4 s	–	$0.000
llama3.2-3b	local	2.1 s	12.3 s	0.9 s	8.1 s	–	$0.000
llama3.2-1b	local	1.6 s	6.1 s	0.6 s	3.3 s	–	$0.000
moondream-1.8b	local	0.6 s	1.2 s	0.1 s	0.2 s	2.0 s	$0.000
gemini-3.1-flash-lite	cloud	1.3 s	3.4 s	2.4 s	1.3 s	2.7 s	$0.173
claude-haiku-5.5	cloud	2.2 s	5.2 s	1.6 s	1.6 s	1.0 s	$0.176
claude-sonnet-5.5	cloud	2.1 s	6.6 s	1.8 s	1.8 s	1.3 s	$2.627

5.3.2 Intent Classification

On the 200 test prompts, written by the dataset authors and never seen in training, the classifier assigned the correct task type to 154 of 160 text prompts (96.3%, 95% CI 92%–98%); with the rule that an attached image means multimodal, 194 of 200 (97.0%). On its own grouped held-out split (paraphrases of a test query never in training) it scores 90.0% (macro-F1 0.899, n =

270). Per task type: coding 100%, writing 100%, search 85%, reasoning 100%, multimodal 100%.

The classifier is trained on 1350 team-written prompts plus the 165 distinct prompts of the public **calibration** split; the public test prompts are never used for training (a test in tests/test_pipeline.py checks this). An earlier version trained on the team's prompts alone scored 166 of 200 (83.0%) on the same test prompts and recognised only 18 of 40 writing prompts, because IFEval instructions contain counts and keywords that look like arithmetic or search. Adding the calibration prompts cut misrouted queries from 34 to 6; balanced routing kept the same accuracy (179/200 before and 179/200 after) at lower cost, because writing prompts now go to the model measured best for writing.

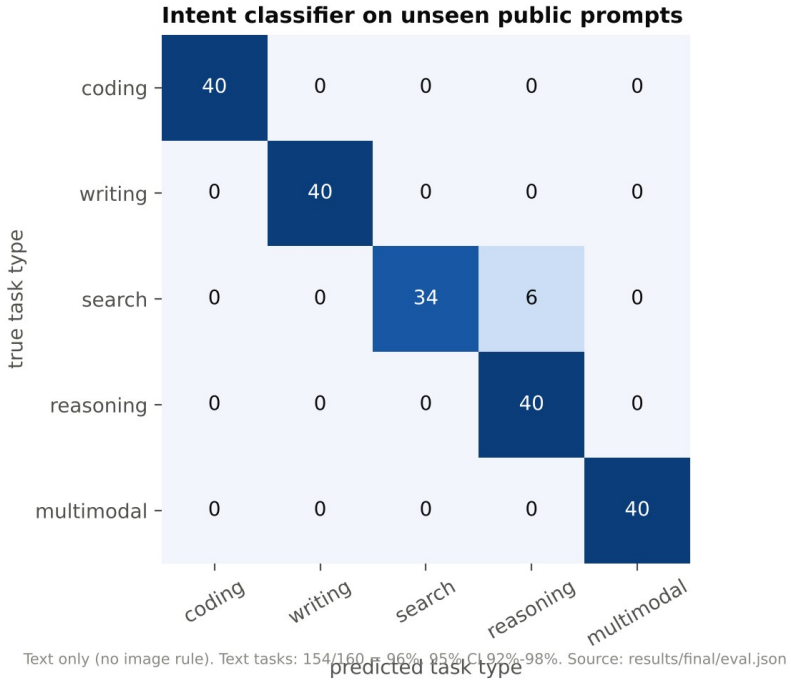


Fig. 13. Intent confusion matrix on the unseen public test prompts (text only)

The team-written prompts used at Review 2 are kept only as a phrasing robustness check, since their authors also wrote the training data: direct 50/50 (CI 93%–100%), casual 47/50 (CI 84%–98%), verbose 49/50 (CI 90%–100%). The "100%" reported for direct prompts at Review 2 is 50/50, whose interval is 93%–100%: in-distribution prompts that the classifier handles well, not evidence of perfection.

5.3.3 Routing Against Single Models

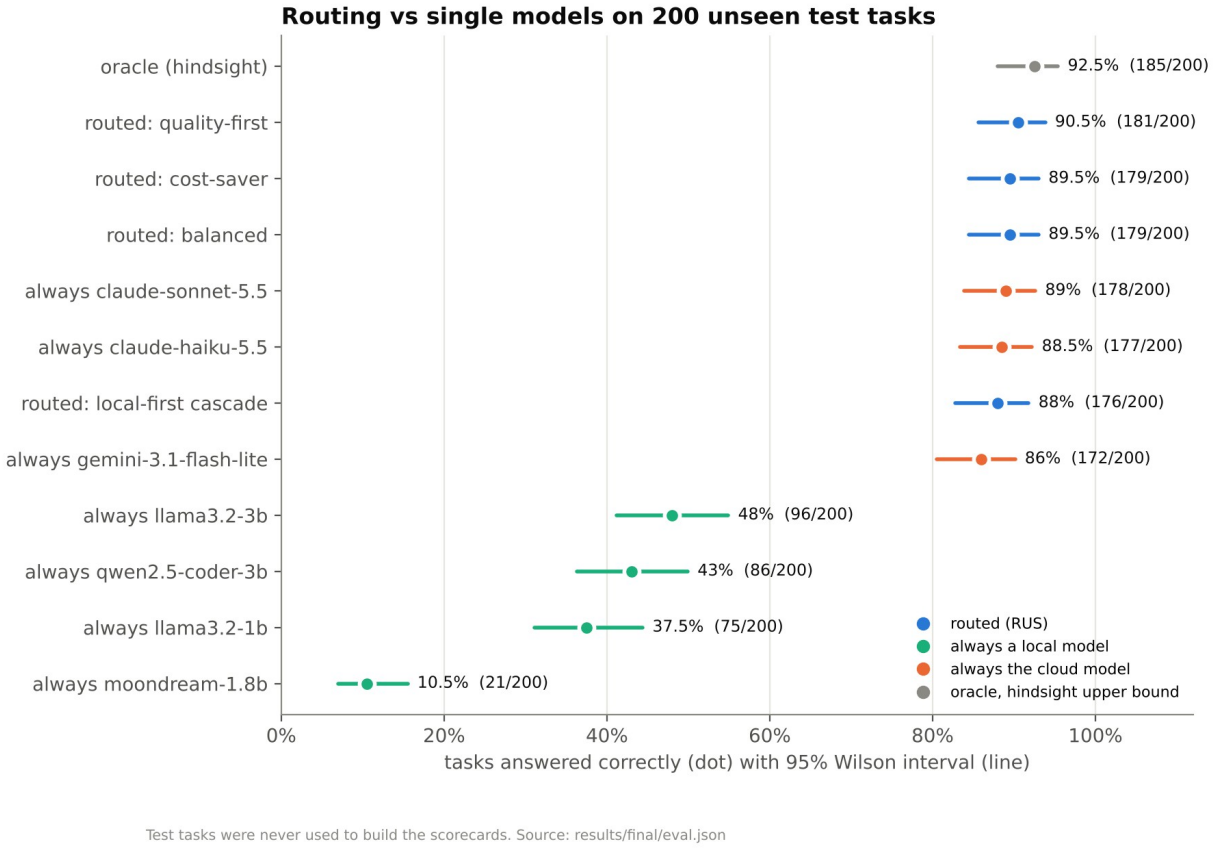


Fig. 14. Accuracy on the 200 unseen test tasks, with 95% Wilson intervals

Table 8. Test-split results for every policy (routed policies highlighted)

Policy	Accuracy (correct/n)	95% CI	USD / 1k queries	INR / 1k	Median latency	On device
oracle (hindsight)	92.5% (185/200)	88% to 95%	$0.111	₹9.80	1.4 s	69%
routed: quality-first	90.5% (181/200)	86% to 94%	$1.04	₹91.61	1.8 s	0%
routed: balanced	89.5% (179/200)	84% to 93%	$0.127	₹11.14	1.5 s	0%
routed: cost-saver	89.5% (179/200)	84% to 93%	$0.127	₹11.14	1.5 s	0%
always claude-sonnet-5.5	89.0% (178/200)	84% to 93%	$2.51	₹220.54	1.9 s	0%
always claude-haiku-5.5	88.5% (177/200)	83% to 92%	$0.169	₹14.89	1.5 s	0%
routed: local-first cascade	88.0% (176/200)	83% to 92%	$0.105	₹9.23	3.1 s	28%
always gemini-3.1-flash-lite	86.0% (172/200)	81% to 90%	$0.170	₹14.95	1.6 s	0%
always llama3.2-3b	48.0% (96/200)	41% to 55%	$0.000	₹0.00	1.6 s	100%

Policy	Accuracy (correct/n)	95% CI	USD / 1k queries	INR / 1k	Median latency	On device
always qwen2.5-coder-3b	43.0% (86/200)	36% to 50%	$0.000	₹0.00	1.4 s	100%
always llama3.2-1b	37.5% (75/200)	31% to 44%	$0.000	₹0.00	1.0 s	100%
always moondream-1.8b	10.5% (21/200)	7% to 16%	$0.000	₹0.00	0.5 s	100%

Under balanced weights the routed system answered 89.5% of test tasks (CI 84% to 93%), above the best local model llama3.2-3b (48.0%): a significant difference (exact McNemar p = 0.000; routed right and llama3.2-3b wrong on 83 tasks, the reverse on 0). Against always using claude-sonnet-5.5 (89.0%) it is not a significant difference (exact McNemar p = 1.00; 6 tasks favour routing, 5 favour claude-sonnet-5.5). The quality-first setting reached 90.5% and the cost-saver 89.5%. The oracle reaches 92.5% while answering 69% of tasks on the device, which shows how much of the pool's ability a better router could still capture. With balanced weights the router sent coding: claude-haiku-5.5 40/40; writing: gemini-3.1-flash-lite 40/40; search: gemini-3.1-flash-lite 34/40, claude-haiku-5.5 6/40; reasoning: claude-haiku-5.5 40/40; multimodal: claude-haiku-5.5 40/40.

The local models receive no traffic from the score alone. llama3.2-3b, the strongest local model, has a lower confidence bound of 65% on arithmetic against 87% for Claude Haiku, while Haiku costs about $0.00014 per query, well under the $0.002 budget. The accuracy term outweighs the small cost saving, so RUS picks a cheap cloud model for every task type. The local-first cascade fixes this without guessing: it kept 27 of 40 coding queries and 30 of 46 arithmetic queries on the device, and overall answered 88.0% (CI 83% to 92%) with 28.5% of queries never leaving the laptop, not a significant difference (exact McNemar p = 0.79; 6 tasks favour routing, 8 favour claude-sonnet-5.5).

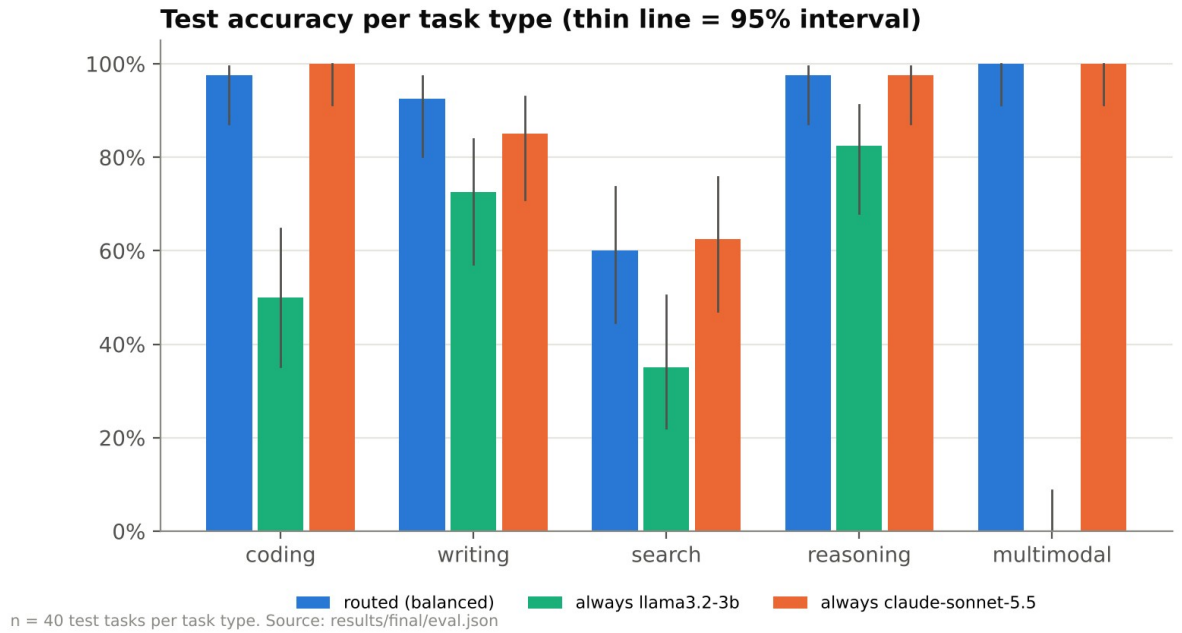


Fig. 15. Test accuracy per task type: routed (balanced) against the best local and the cloud model

Table 9. Paired significance tests on the test split

Comparison	Routed right, other wrong	Other right, routed wrong	p (exact McNemar)	Verdict
routed: balanced vs always qwen2.5-coder-3b	93	0	0.000	routing better
routed: balanced vs always llama3.2-3b	83	0	0.000	routing better
routed: balanced vs always llama3.2-1b	104	0	0.000	routing better
routed: balanced vs always moondream-1.8b	158	0	0.000	routing better
routed: balanced vs always gemini-3.1-flash-lite	7	0	0.016	routing better
routed: balanced vs always claude-haiku-5.5	5	3	0.727	no significant difference
routed: balanced vs always claude-sonnet-5.5	6	5	1.000	no significant difference
routed: quality-first vs always qwen2.5-coder-3b	95	0	0.000	routing better
routed: quality-first vs always llama3.2-3b	85	0	0.000	routing better
routed: quality-first vs always llama3.2-1b	106	0	0.000	routing better
routed: quality-first vs always moondream-1.8b	160	0	0.000	routing better
routed: quality-first vs always gemini-3.1-flash-lite	10	1	0.012	routing better
routed: quality-first vs always claude-haiku-5.5	6	2	0.289	no significant difference
routed: quality-first vs always claude-sonnet-5.5	5	2	0.453	no significant difference
routed: cost-saver vs always qwen2.5-coder-3b	93	0	0.000	routing better
routed: cost-saver vs always llama3.2-3b	83	0	0.000	routing better
routed: cost-saver vs always llama3.2-1b	104	0	0.000	routing better
routed: cost-saver vs always moondream-1.8b	158	0	0.000	routing better
routed: cost-saver vs always gemini-3.1-flash-lite	7	0	0.016	routing better
routed: cost-saver vs always claude-haiku-5.5	5	3	0.727	no significant difference
routed: cost-saver vs always claude-sonnet-5.5	6	5	1.000	no significant difference

5.3.4 Cost

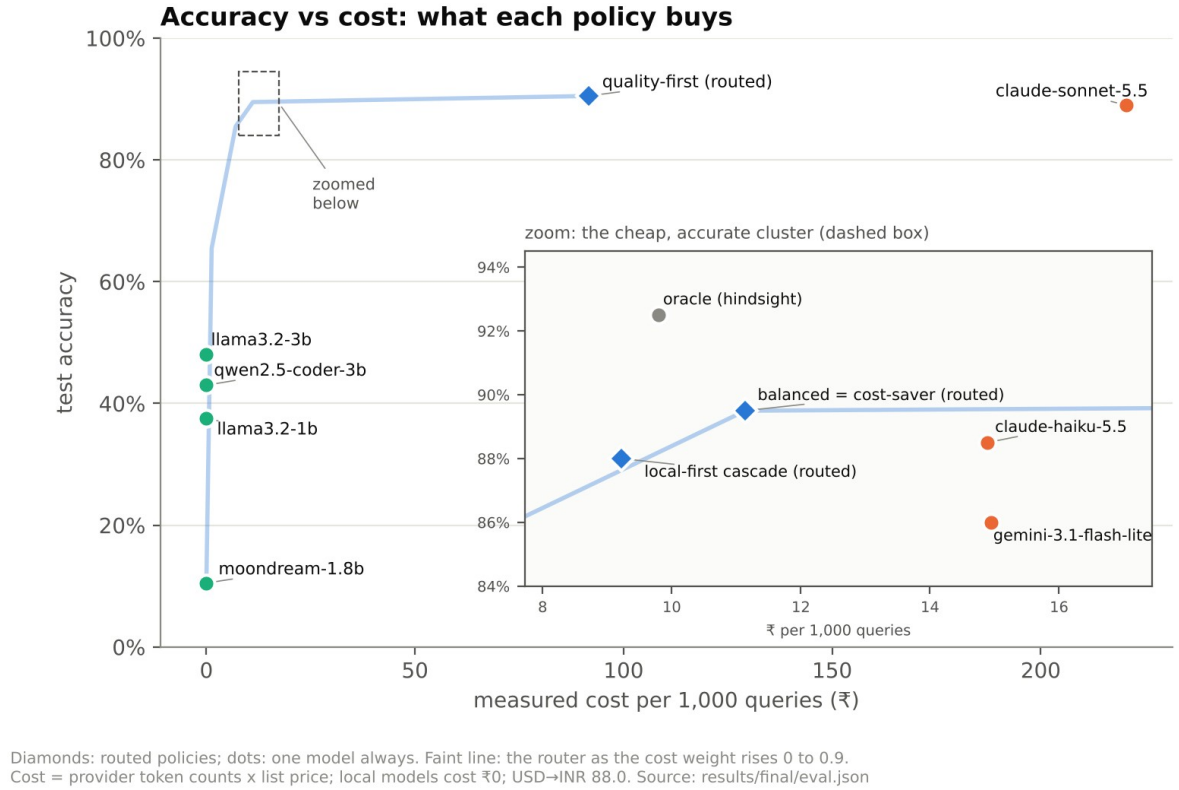


Fig. 16. Accuracy against measured cost per 1,000 queries; the faint line is the router as the cost weight rises from 0 to 0.9

Always using claude-sonnet-5.5 costs $2.51 (₹220.54) per 1,000 queries at the measured token counts. Balanced routing costs $0.127 (₹11.14), a 95% reduction, answering 0% of queries on the device, for an accuracy difference of +0.5 points. The cost-saver setting removes 95% of the API cost for 0.5 points; quality-first spends 42% of the cloud cost for +1.5 points. The local-first cascade costs $0.105 per 1,000 queries (96% below claude-sonnet-5.5) while answering 28.5% of queries on the device. The weights and the cascade let a user choose the point on this curve that suits them, and the curve itself is measured, not assumed.

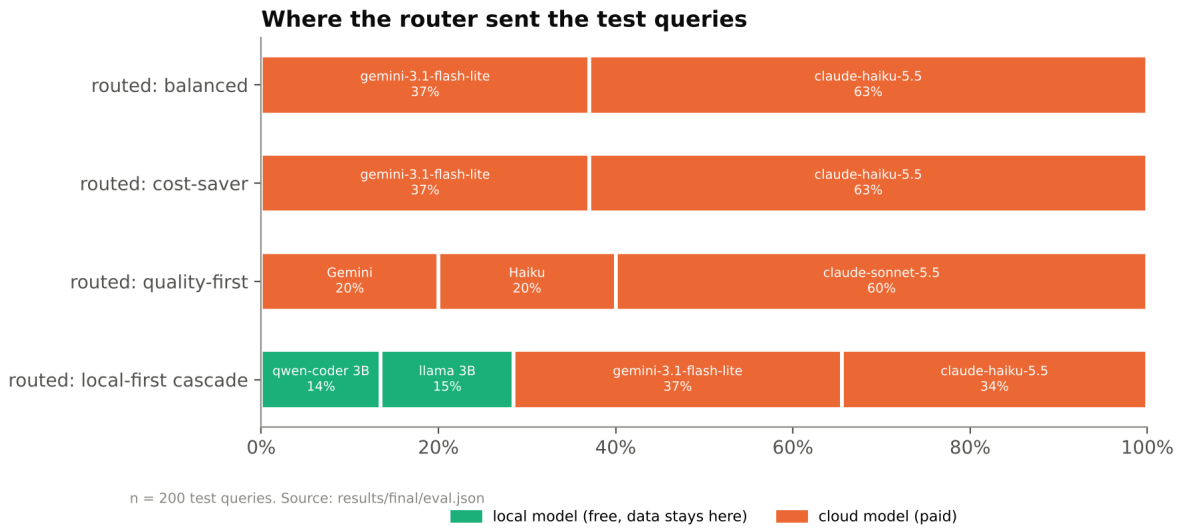


Fig. 17. Share of test queries each routing setting sent to each model

5.3.5 Stability of Cloud Answers

Local models run at temperature 0 with a fixed seed and give the same answer every time. Hosted models can vary between calls, so every cloud model answered the 200 test tasks a second time.

Table 10. Second run of every cloud model on the test split

Model	Run 1	Run 2	Same mark	Identical text	McNemar p
claude-haiku-5.5	88.5% (177/200)	86.5% (173/200)	96.0%	38%	0.29
claude-sonnet-5.5	89.0% (178/200)	89.0% (178/200)	99.0%	42%	1.00
gemini-3.1-flash-lite	85.9% (170/198)	85.9% (170/198)	100.0%	100%	1.00

A difference between the runs that is not significant means a single run is a fair estimate of each model; the scorecards still carry the sampling interval, which is wider than the run-to-run variation.

5.3.6 Ablation of Memory and Retrieval

The five builds share one pipeline; the non-routing builds always use llama3.2-3b, and routing chooses among the local models (qwen2.5-coder-3b, llama3.2-3b, llama3.2-1b, moondream-1.8b). The memory benchmark has a simulated user state facts over four sessions and then asks 30 recall questions in a new session; the retrieval benchmark asks 36 questions about a fictional company whose facts no model can know without its documents.

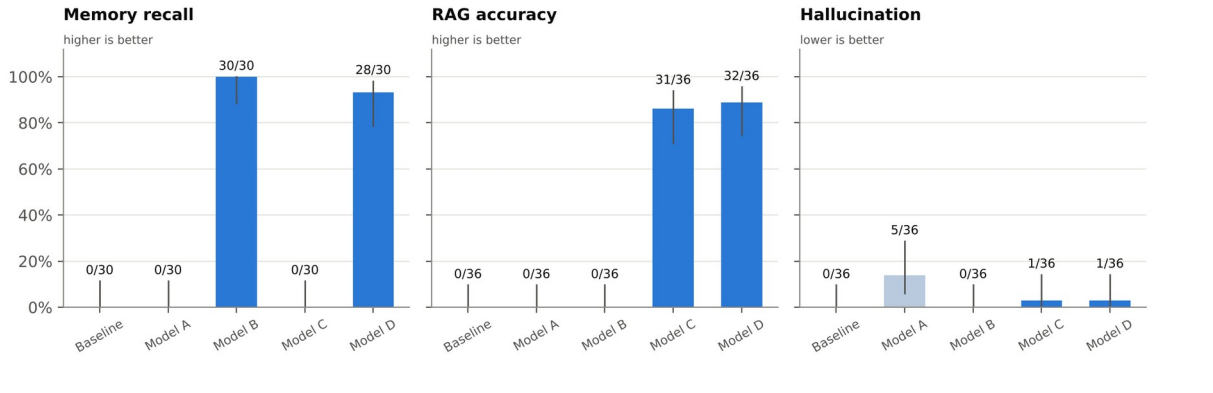


Fig. 18. Ablation: memory recall, retrieval accuracy and hallucination per build, with 95% intervals

Table 11. Ablation results (Model D highlighted)

Build	Switches	Memory recall	RAG accuracy	Hallucination	Abstain	Cited
Baseline	none	0/30 (0%–11%)	0% (0%–10%)	0% (0%–10%)	100%	0%
Model A	routing	0/30 (0%–11%)	0% (0%–10%)	14% (6%–29%)	86%	0%
Model B	memory	30/30 (89%–100%)	0% (0%–10%)	0% (0%–10%)	100%	0%

Build	Switches	Memory recall	RAG accuracy	Hallucination	Abstain	Cited
Model C	RAG	0/30 (0%–11%)	86% (71%–94%)	3% (0%–14%)	11%	94%
Model D	routing + memory + RAG	28/30 (79%–98%)	89% (75%–96%)	3% (0%–14%)	8%	97%

5.3.7 Privacy Check

Model D was run on the memory benchmark with the **full** pool, including gemini-3.1-flash-lite, claude-haiku-5.5, claude-sonnet-5.5, and the default private-context policy. Of 54 requests, 0 went to a cloud model and **0 carried personal context**; recall was 93% (CI 79%–98%). Every cloud request in normal use is listed in the egress log with its size and a hash of its content, never its text.

5.4 Prototype Demonstration

The Streamlit app shows every decision: the predicted task type and its probabilities, the full Routing Utility Score table with each measured input, whether the answer left the computer, its token counts and cost, the memory items recalled, and the passages retrieved. The screenshots below were captured from the running app on the reference laptop.

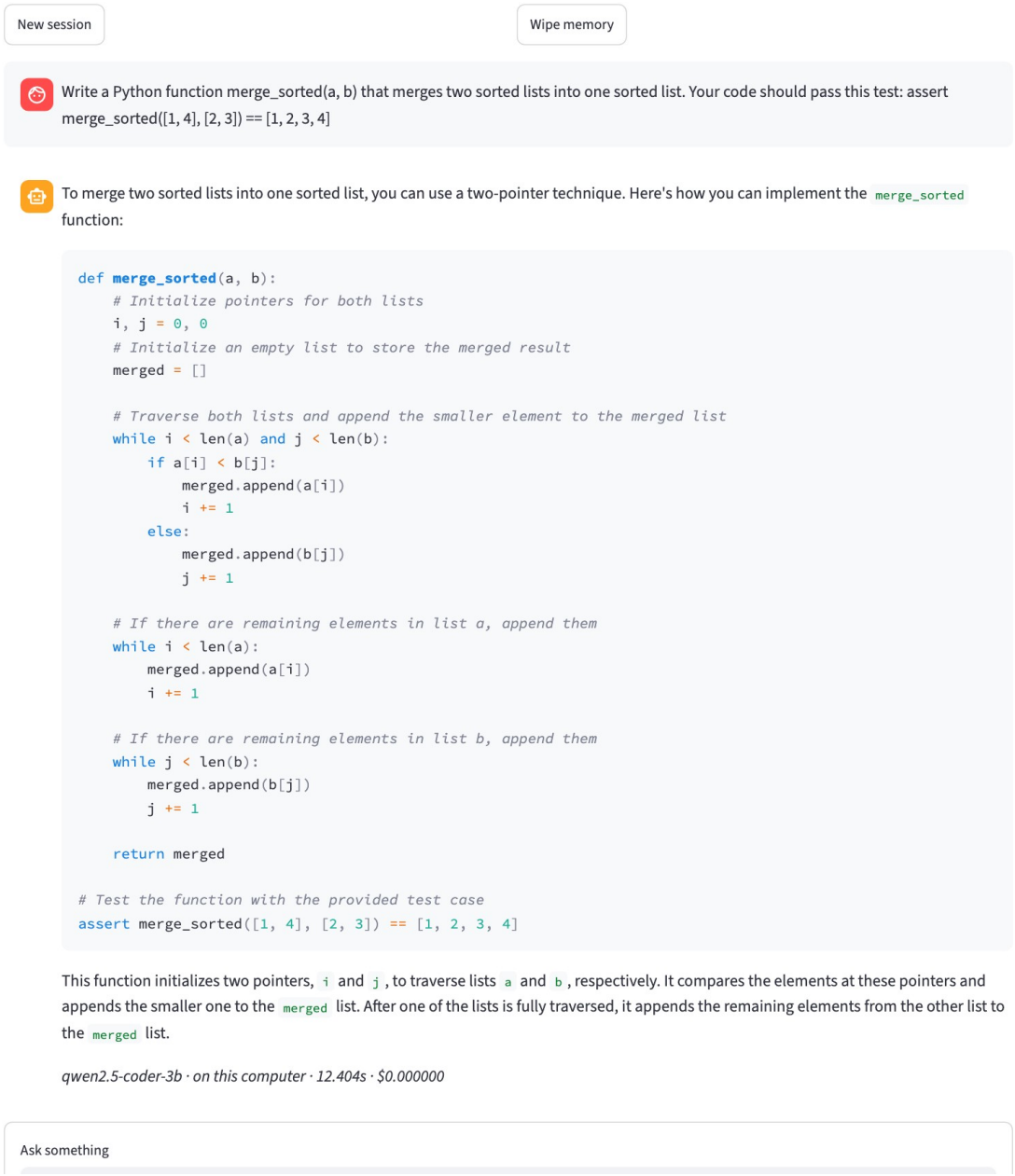


Fig. 19. A coding query with an example test in the prompt: the local coder's answer passes that test, so it is kept on the laptop at zero cost

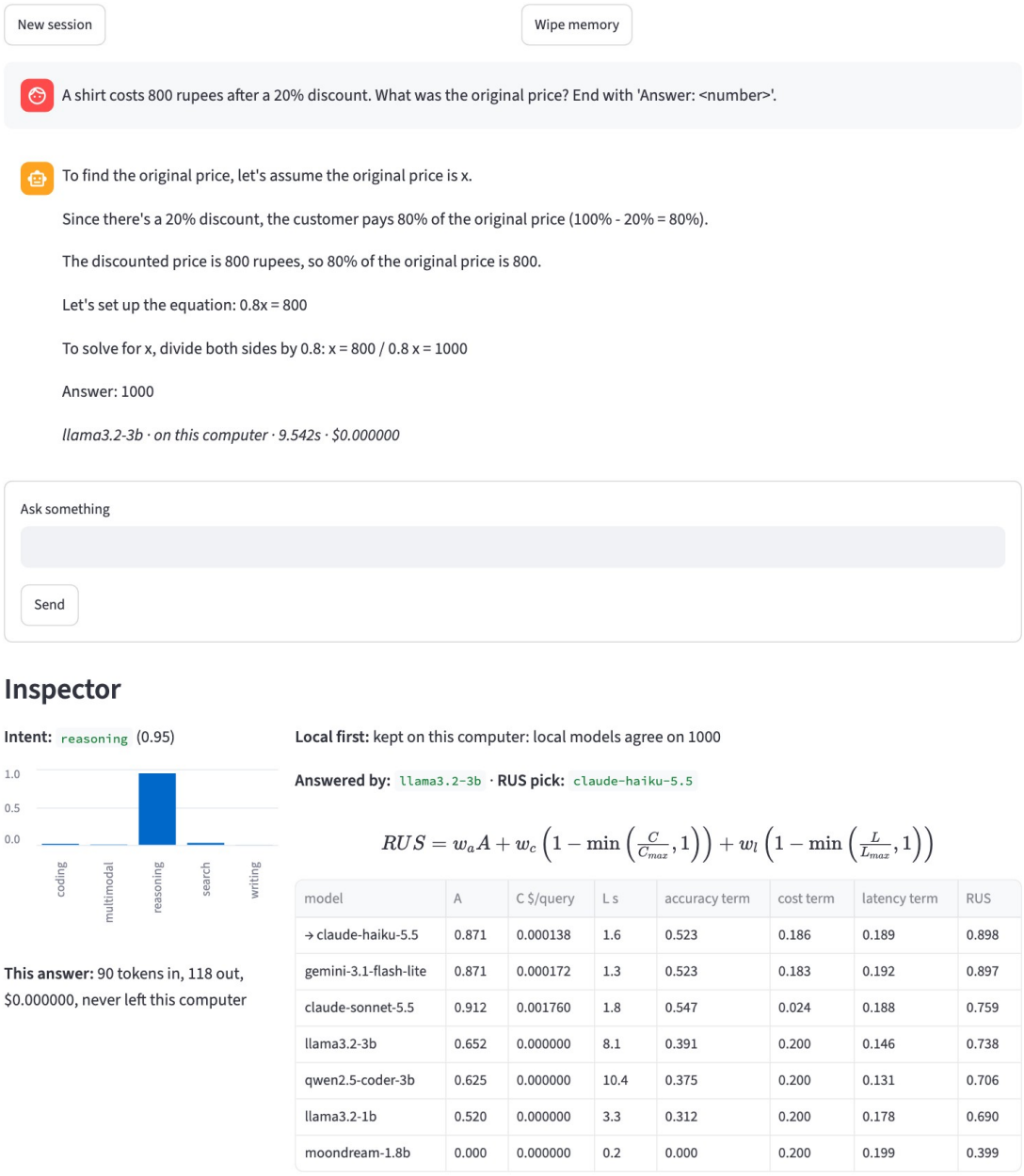


Fig. 20. An arithmetic query: two local models agree on 1000, so the answer never leaves the laptop; the table shows that RUS alone would have picked Claude Haiku

New session

Wipe memory

Who wrote the novel Midnight's Children?

Salman Rushdie wrote *Midnight's Children*.

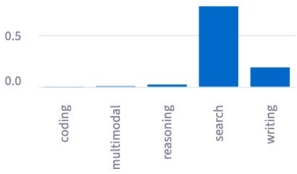
gemini-3.1-flash-lite · cloud · 2.241s · $0.000026

Ask something

Send

Inspector

Intent: `search` (0.78)



This answer: 45 tokens in, 10 out, $0.000026, sent to cloud

Answered by: `gemini-3.1-flash-lite` · **RUS pick:** `gemini-3.1-flash-lite`

$$RUS = w_a A + w_c \left(1 - \min\left(\frac{C}{C_{max}}, 1\right)\right) + w_l \left(1 - \min\left(\frac{L}{L_{max}}, 1\right)\right)$$

model	A	C $/query	L s	accuracy term	cost term	latency term	RUS
→ gemini-3.1-flash-lite	0.375	0.000041	2.4	0.225	0.196	0.184	0.605
claude-haiku-5.5	0.329	0.000096	1.6	0.198	0.190	0.189	0.577
claude-sonnet-5.5	0.470	0.001755	1.8	0.282	0.025	0.188	0.495
llama3.2-3b	0.142	0.000000	0.9	0.085	0.200	0.194	0.479
llama3.2-1b	0.105	0.000000	0.6	0.063	0.200	0.196	0.459
qwen2.5-coder-3b	0.071	0.000000	0.8	0.042	0.200	0.195	0.437
moondream-1.8b	0.000	0.000000	0.1	0.000	0.200	0.199	0.399

A = accuracy lower 95% bound on this task type · C = measured cost per query · L = median latency

› System prompt sent to the model

Fig. 21. A factual query under balanced weights: Gemini 3.1 Flash-Lite has the highest RUS for search and answers it

New session

Wipe memory

Who wrote the novel Midnight's Children?

Salman Rushdie wrote *Midnight's Children*.

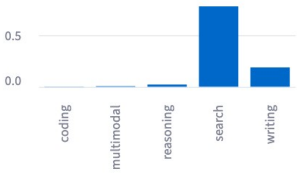
gemini-3.1-flash-lite · cloud · 2.553s · $0.000026

Ask something

Send

Inspector

Intent: `search` (0.78)



This answer: 45 tokens in, 10 out, $0.000026, sent to cloud

Answered by: `gemini-3.1-flash-lite` · **RUS pick:** `gemini-3.1-flash-lite`

$$RUS = w_a A + w_c \left(1 - \min\left(\frac{C}{C_{max}}, 1\right)\right) + w_l \left(1 - \min\left(\frac{L}{L_{max}}, 1\right)\right)$$

model	A	C $/query	L s	accuracy term	cost term	latency term	RUS
→ gemini-3.1-flash-lite	0.375	0.000041	2.4	0.113	0.588	0.092	0.792
claude-haiku-5.5	0.329	0.000096	1.6	0.099	0.571	0.095	0.765
llama3.2-3b	0.142	0.000000	0.9	0.043	0.600	0.097	0.740
llama3.2-1b	0.105	0.000000	0.6	0.032	0.600	0.098	0.729
qwen2.5-coder-3b	0.071	0.000000	0.8	0.021	0.600	0.097	0.719
moondream-1.8b	0.000	0.000000	0.1	0.000	0.600	0.100	0.700
claude-sonnet-5.5	0.470	0.001755	1.8	0.141	0.073	0.094	0.308

A = accuracy lower 95% bound on this task type · C = measured cost per query · L = median latency

› System prompt sent to the model

Fig. 22. The same query with the cost weight raised to 0.6: every term is re-weighted and the local models overtake Claude Sonnet, but Gemini still wins because its cost per query is close to zero

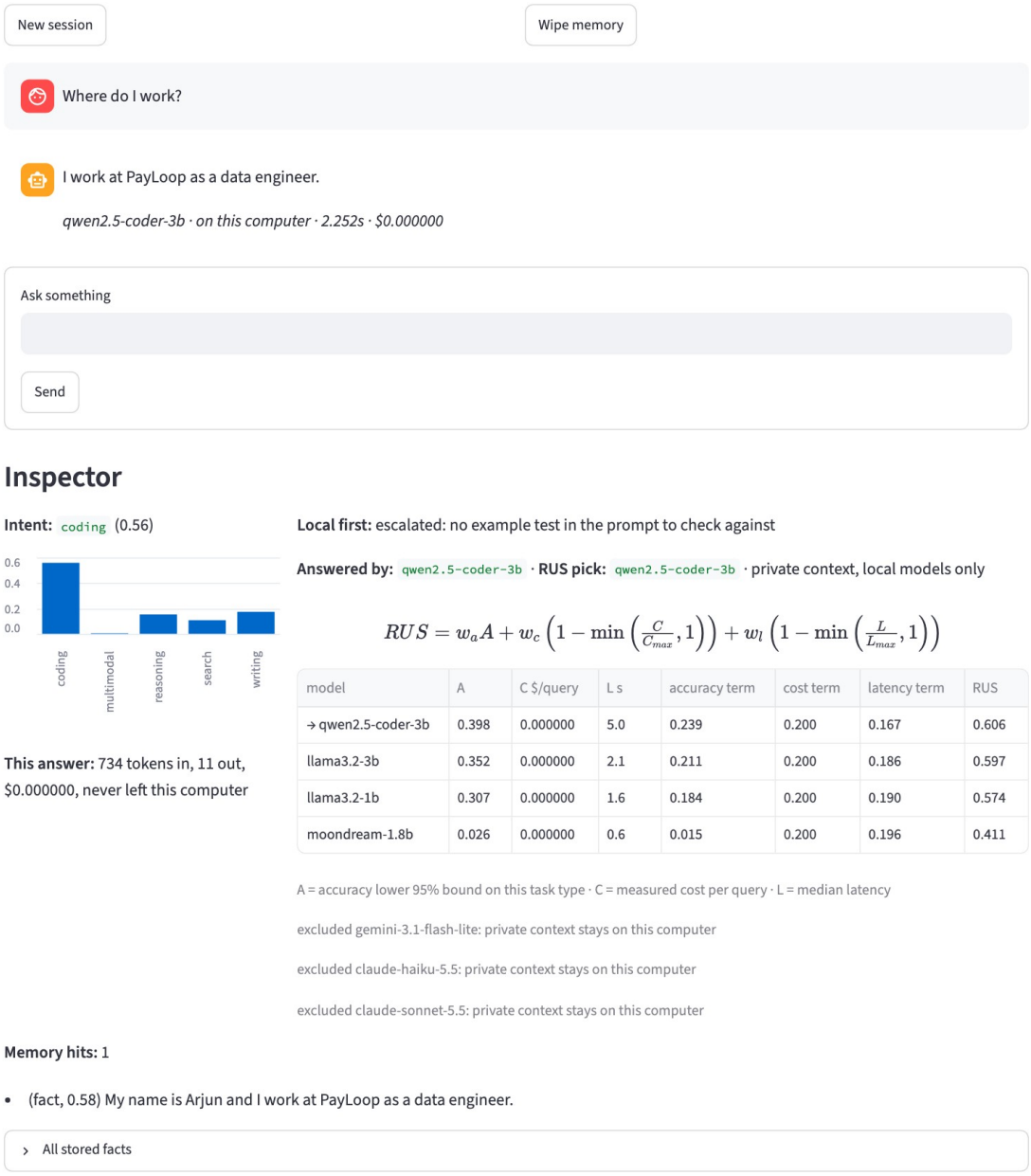


Fig. 23. A fact from an earlier session is recalled. The classifier mislabels the question as coding (one of its known errors), but because the query carries a memory, every cloud model is excluded and the answer stays on the laptop

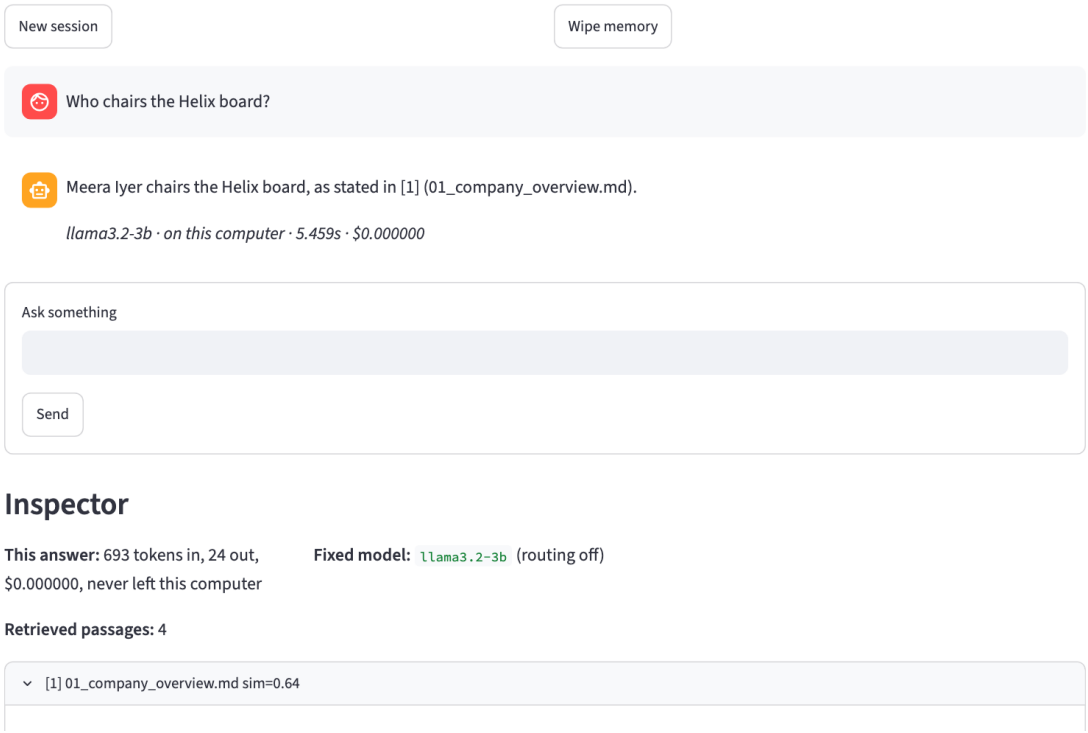


Fig. 24. A closed-domain question answered from the user's own document, with the source cited as [1]; the four retrieved passages are listed below the answer

claude-haiku-5.5 · claude-haiku-5-5 · measured on Linux x86_64 vm, 2026-10-20

task type	n	correct	accuracy	95% CI	median s	$ per query
coding	40	39	98%	87% to 100%	2.2	0.000132
writing	40	34	85%	71% to 93%	5.2	0.000474
search	40	19	48%	33% to 62%	1.6	0.000096
reasoning	40	39	98%	87% to 100%	1.6	0.000138
multimodal	40	40	100%	91% to 100%	1.0	0.000038

claude-sonnet-5.5 · claude-sonnet-5-5 · measured on Linux x86_64 vm, 2026-10-20

task type	n	correct	accuracy	95% CI	median s	$ per query
coding	40	40	100%	91% to 100%	2.1	0.001618
writing	40	38	95%	84% to 99%	6.6	0.007281
search	40	25	62%	47% to 76%	1.8	0.001755
reasoning	40	40	100%	91% to 100%	1.8	0.001760
multimodal	40	40	100%	91% to 100%	1.3	0.000720

gemini-3.1-flash-lite · gemini-3.1-flash-lite · measured on Linux x86_64 vm, 2026-10-20

task type	n	correct	accuracy	95% CI	median s	$ per query
coding	40	35	88%	74% to 95%	1.3	0.000123
writing	40	38	95%	84% to 99%	3.4	0.000242
search	40	21	52%	38% to 67%	2.4	0.000041
reasoning	40	39	98%	87% to 100%	1.3	0.000172
multimodal	40	40	100%	91% to 100%	2.7	0.000286

llama3.2-1b · llama3.2:1b · measured on Linux x86_64 vm, 2026-10-20

task type	n	correct	accuracy	95% CI	median s	$ per query
coding	40	18	45%	31% to 60%	1.6	0.000000
writing	40	25	62%	47% to 76%	6.1	0.000000
search	40	8	20%	10% to 35%	0.6	0.000000
reasoning	40	27	68%	52% to 80%	3.3	0.000000
multimodal	0	-	-	cannot do this	-	-

llama3.2-3b · llama3.2:3b · measured on Linux x86_64 vm, 2026-10-20

Fig. 25. The Scorecards tab: what each model was measured to do

5.5 Discussion

- **Routing is graded on evidence it did not see.** Scorecards come from the calibration split; every routing number comes from the test split, against every fixed model, with paired tests.
- **Cheap cloud models changed the economics.** At about $0.0001 per query, small cloud models cost almost nothing and are far more accurate than 1–3B local models, so a pure cost-accuracy score never prefers the laptop. The saving against a premium model is large (95% against claude-sonnet-5.5) and comes at no measured loss of accuracy.
- **Local-first needs a check, not a guess.** Keeping an answer local only when a reference-free check passes moved 28.5% of queries onto the laptop. The oracle shows 69% could be answered locally, so better checks are the clearest path to more on-device answers.

- **Memory is the only source of cross-session recall, and retrieval the main source of closed-domain accuracy;** the full build keeps both.
- **Private context stays local by construction,** and the privacy check confirms it on real runs.
- **Every grounded answer now points to its source.** Small local models often answer from the passages but drop the [n] markers the prompt asks for: the model wrote its own markers in 20 of 36 build-C answers and 31 of 36 build-D answers. For the rest the pipeline lists the passages the answer actually draws on (pipeline.attach_sources), so 34 and 35 of 36 answers point to a source; an answer that uses no passage, such as "I don't know", gets none.

5.5.1 What Changed Since Review 2

Table 12. Changes made in response to the Review 2 feedback

Review 2	Final
Per-model accuracy was a hand-entered prior (0.95, 0.90 …)	Measured on 200 public calibration tasks; lower 95% bound used
Cost and latency were unit-less guesses in [0, 1]	USD per query from token counts; median seconds on this laptop
'Routing gain +0.083' compared the router's score with itself	Routing graded on a disjoint test split against every model, with McNemar tests
50 team-written tasks, 10 per type, no intervals ('100%')	400 public tasks; every rate with n and a 95% interval
Roles named GPT / Claude / Gemini / Sonar played by small local models	Real model names; any model added through models.yaml
Diagrams drawn separately from the code	Diagrams generated from the code; charts generated from logged results

5.5.2 Positioning Against Perplexity AI

Table 13. Comparison with the closest production system

Aspect	Perplexity AI	This project
Routing rule	Undisclosed	Published closed-form score; every input measured
Why a model was chosen	Not shown	Full scoring table for every query
Where data goes	Provider's servers	Memory and documents stay on the device; egress logged
Models	Fixed commercial set	Any local or hosted model, added by configuration
Evidence	No published ablation	Public-benchmark evaluation, ablation, intervals and tests
Retrieval source	Live web	The user's own documents (web search is future work)

Perplexity is stronger at live web search, model scale and production reliability. This project contributes a router whose decisions can be inspected and reproduced, measured cost savings, and local-first handling of personal data.

5.6 Limitations and Future Work

- **Sample size.** 40 test tasks per type give intervals of roughly ±15 points per type; the 200-task totals are tighter. Larger splits would sharpen per-type conclusions.
- **Routing granularity.** The score decides per task type; the cascade adds per-query checks only for code with an example test and for arithmetic. Checks for writing (verifiable instructions) and factual answers (agreement or retrieval) would capture more of the oracle's headroom.
- **Model scale.** The local models are 1B–3B parameters to fit 8 GB of memory; 7B–8B models on 16 GB machines would shift more traffic on-device.
- **Graders.** Code-based grading is strict and can mark a correct but differently formatted answer wrong; every grade is logged with its reason so it can be audited.
- **Intent classifier.** A TF-IDF model: it still misroutes 6 of 200 public test prompts, mostly factual questions phrased like arithmetic. A small embedding-based classifier would handle unusual phrasings better.
- **Future work:** per-query uncertainty-based escalation, live web retrieval, more providers' price tables, and a conference paper from these results.

6. REFERENCES

[1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020.

[2] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” ICLR, 2023.

[3] C. Packer et al., “MemGPT: Towards LLMs as Operating Systems,” arXiv:2310.08560, 2023.

[4] L. Chen, M. Zaharia and J. Zou, “FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance,” arXiv:2305.05176, 2023.

[5] D. Ding et al., “Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing,” ICLR, 2024.

[6] I. Ong et al., “RouteLLM: Learning to Route LLMs with Preference Data,” arXiv:2406.18665, 2024.

[7] ByteByteGo, “How Perplexity Built an AI-Powered Search Engine,” 2025.

[8] J. Austin et al., “Program Synthesis with Large Language Models” (MBPP), arXiv:2108.07732, 2021.

[9] K. Cobbe et al., “Training Verifiers to Solve Math Word Problems” (GSM8K), arXiv:2110.14168, 2021.

[10] K. Lee, M.-W. Chang and K. Toutanova, “Latent Retrieval for Weakly Supervised Open Domain Question Answering,” ACL, 2019; T. Kwiatkowski et al., “Natural Questions,” TACL, 2019.

[11] J. Zhou et al., “Instruction-Following Evaluation for Large Language Models” (IFEval), arXiv:2311.07911, 2023.

[12] E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference,” J. American Statistical Association, 22(158), 1927.

[13] Q. McNemar, “Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages,” Psychometrika, 12(2), 1947.

[14] W. Wang et al., “Augmenting Language Models with Long-Term Memory” (LongMem), NeurIPS, 2023.

[15] Ollama, “Get up and running with large language models,” https://ollama.com.

[16] Google, “Gemini API pricing,” https://ai.google.dev/gemini-api/docs/pricing (accessed Oct. 2026).

[17] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” JMLR, 12, 2011.